

Gradient descent

Foundations of AI

Carlos Cotrini

UZH CAS Gamechanger AI

26 September 2026

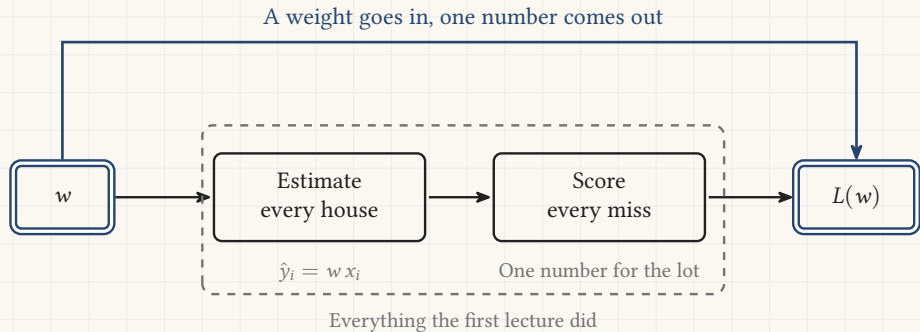
1

Section 1

From the loop to a curve

One round, as a function

One turn of the loop from the first lecture, with the machinery boxed up.



Scoring a guess

Two flats, both 100 square metres, priced differently. One guess: $w = 25\,000$ per square metre.

Flat	Price	Estimate	Miss
1	2 000 000	2 500 000	-500 000
2	3 000 000	2 500 000	+500 000

The average miss

$$\frac{(-500\,000) + (+500\,000)}{2} = 0$$

Zero, and both flats are half a million out.

The average squared miss

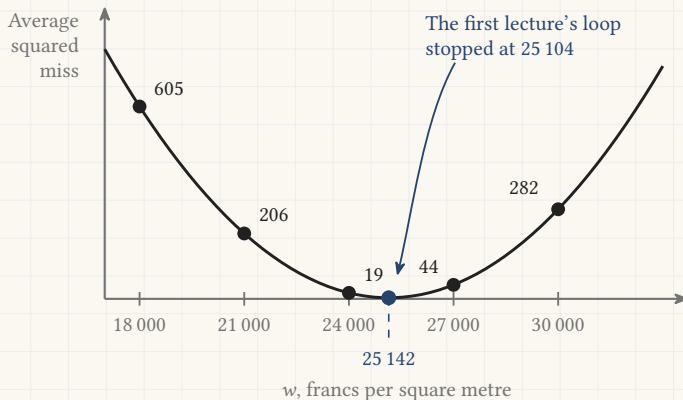
$$\frac{(-500\,000)^2 + (+500\,000)^2}{2} = 500\,000^2$$

Zero only when every miss is zero.

Five houses, one knob

The five houses of the first lecture, priced by area alone. Pick a w , score it, and plot the score.

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113



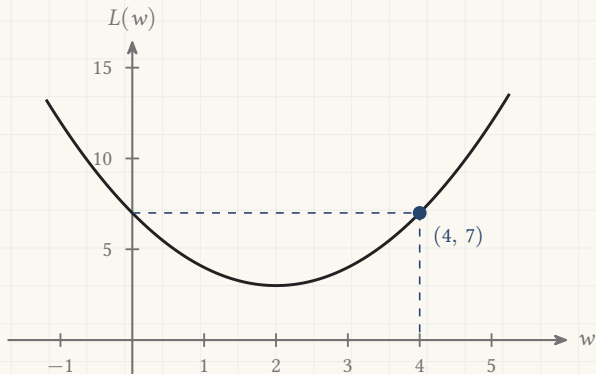
2

Section 2

Reading a curve

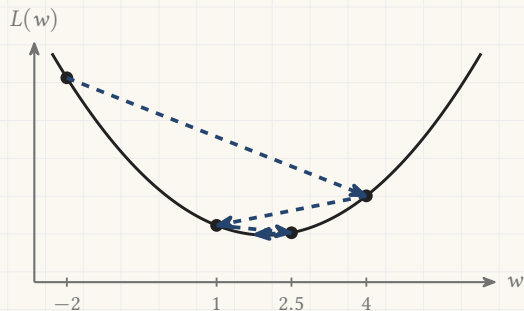
The loss curve

$$L(w) = w^2 - 4w + 7$$



Reading the curve

Stand somewhere. Read the height. Decide where to go next.



w	$L(w)$	Update
-2	19	→
4	7	←
1	4	→
2.5	3.25	←

Each arrow is half the length of the one above it, and points the other way.

3

Section 3

The grasshopper game

The grasshopper game

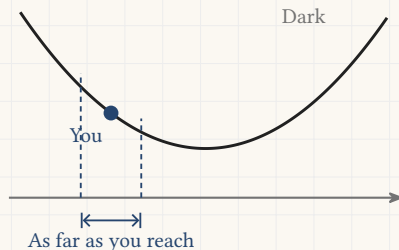
You are a grasshopper on a hillside in the dark. You want the lowest point.

- You know where you are standing.
- You can feel the ground under your feet, as far as your legs reach.
- You cannot see the hillside.

Play it here:

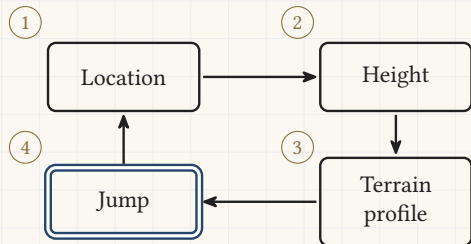


https://carloscotrini.github.io/FAI-PAGE/viz/grasshopper/cx_grasshopper.html

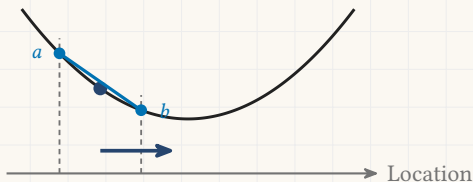


One round of the game

The 09:00 loop again, with the grasshopper's own four words.



Where you stand, how high it is,
how the ground tilts, where you land

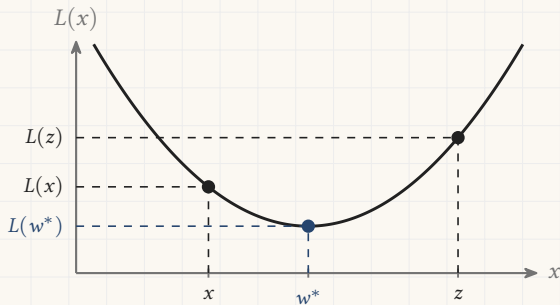


The tilt between the two feet.

The game, written down

$$\begin{array}{ccc} L : & x & \mapsto & L(x) \\ & \uparrow & & \uparrow \\ & \text{Location} & & \text{Height} \end{array}$$

Find w^* in $\mathbb{R}$ such that $L(w^*) \leq L(x)$
for every x in $\mathbb{R}$.



Nothing in that sentence mentions a hillside.

The gradient

Stand anywhere on the hillside. What you can feel underfoot is the tilt of the ground.



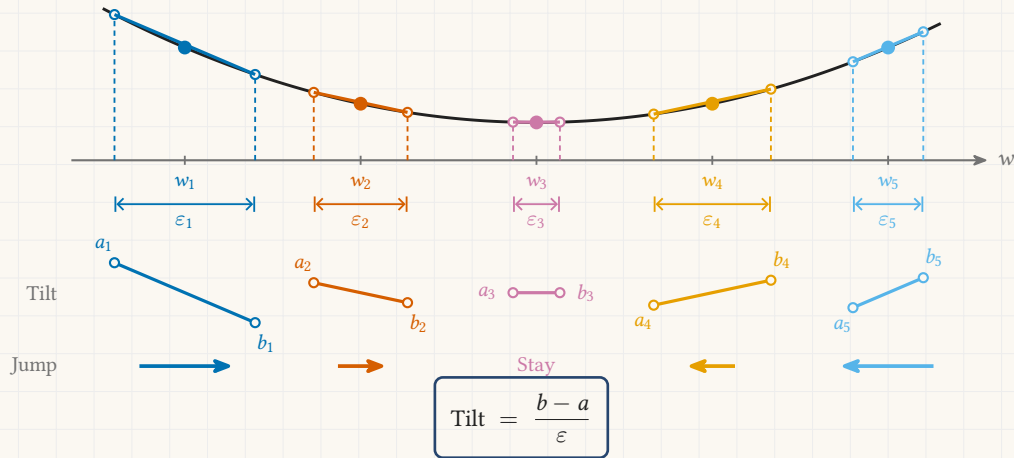
4

Section 4

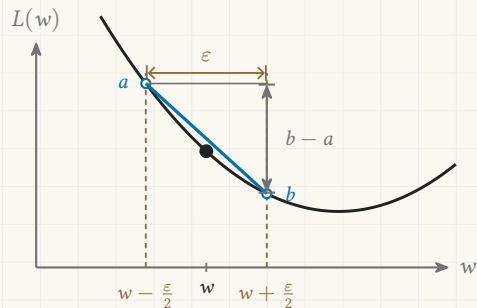
Reading the ground under your feet*

Five players, five windows*

Each player feels the ground as far as their own legs reach. * Optional, and not covered in the lecture.



The window, and its name*



$$a = L\left(w - \frac{\epsilon}{2}\right)$$

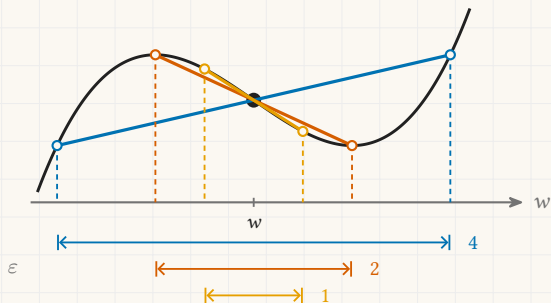
$$b = L\left(w + \frac{\epsilon}{2}\right)$$

$$\begin{aligned}\nabla_{\epsilon} L(w) &= \frac{b - a}{\epsilon} \\ &= \frac{L\left(w + \frac{\epsilon}{2}\right) - L\left(w - \frac{\epsilon}{2}\right)}{\epsilon}\end{aligned}$$

Its name is the **central difference quotient** of L at w over a window of width ϵ .

Which window is the right one*

A different L , one that is not a bowl, and one place to stand on it.



$$\varepsilon = 4 \quad \nabla_4 L(w) = 1$$

$$\varepsilon = 2 \quad \nabla_2 L(w) = -2$$

$$\varepsilon = 1 \quad \nabla_1 L(w) = -2.75$$

The widest window says the ground rises to the right. It does not.

$$\varepsilon \rightarrow 0$$



$$\nabla L(w) = -3$$

The gradient*

Make the window arbitrarily narrow. For the functions this course meets, the answers settle on one number.

$$\nabla L(w) = \lim_{\varepsilon \rightarrow 0} \frac{L(w + \frac{\varepsilon}{2}) - L(w - \frac{\varepsilon}{2})}{\varepsilon}$$

- It is one number, and it is a tilt: how fast L climbs at w , and in which direction.
- A negative gradient means the ground falls away to the right, so the lower place is to the right.
- The steeper the tilt, the further the lower place is likely to be.

Those three lines are the whole of the update rule, and the next frame writes them down.

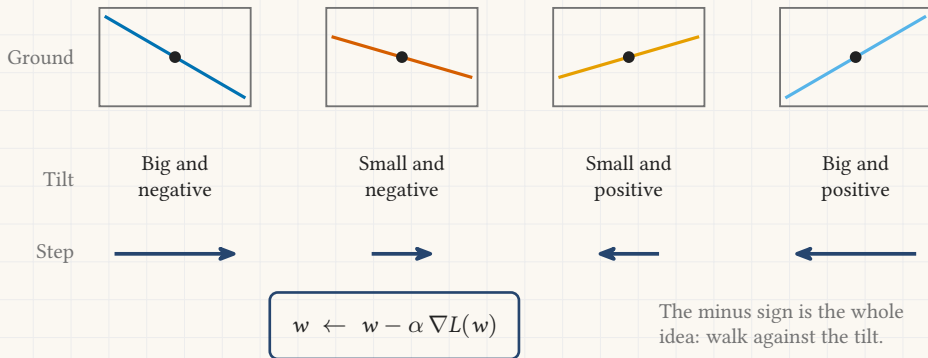
5

Section 5

Gradient descent*

From tilt to step*

The same patch of ground, tilted four ways. * Optional, and not covered in the lecture.



The algorithm*

The 09:00 loop, with the gradient in place of the miss.

```
def gradient_descent(L, w, alpha, eps, tol, steps):  
    for n in range(steps):  
        a = L(w - eps/2)      # height at the left of the window  
        b = L(w + eps/2)      # height at the right of the window  
        g = (b - a) / eps      # the tilt  
        if abs(g) < tol:      # flat enough: you are at the bottom  
            break  
        w = w - alpha * g      # the step: walk against the tilt  
    return w
```

Our curve, from a bad guess:

```
def L(w):  
    return w**2 - 4*w + 7  
gradient_descent(L, -2, 0.75, 0.1, 0.01, 20)
```

eps is the window and tol is how flat counts as flat. alpha is still unchosen.

A run of gradient descent*

$L(w) = w^2 - 4w + 7$, starting at $w = -2$, with $\alpha = 0.75$.

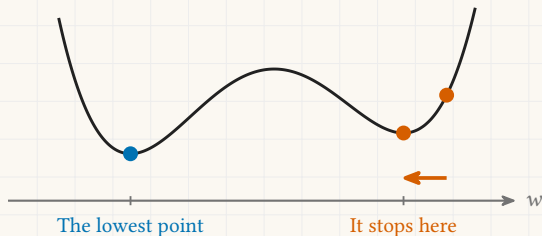
w	$L(w)$	$\nabla L(w)$	$-\alpha \nabla L(w)$	Step
-2	19	-8	+6	
4	7	4	-3	
1	4	-2	+1.5	
2.5	3.25	1	-0.75	
1.75	3.0625	-0.5	+0.375	

$w^* = 2$, where $L(w^*) = 3$.

The first four rows are the four the room read off by eye. The arrows were the gradient.

Where gradient descent stops*

The rule is local. It only ever knows the ground under its own feet.



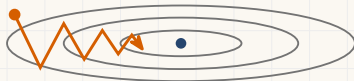
- Every step is downhill.
- The walk stops where the ground is flat.
- Flat is not the same as lowest.

In practice this is answered by starting the walk several times from several places and keeping the best result.

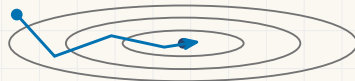
Momentum, 1964*

* Optional. Nothing later in the lecture depends on this frame or the next.

A long narrow valley is where the plain rule is worst. The steep direction sets the step, so the walk crosses the valley instead of following it.



The plain rule
Eight steps, still short of the bottom



With momentum
Four steps, and it is there

$$v \leftarrow \beta v + \nabla L(w)$$

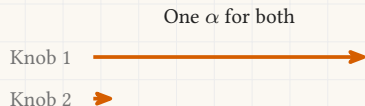
$$w \leftarrow w - \alpha v$$

v remembers the direction you were already going, at strength β , usually about 0.9. Across the valley it keeps changing sign and cancels; along the floor it keeps its sign and adds up.

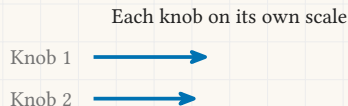
Adam, 2015*

* Optional, as the previous frame.

The other failure of one step size: two knobs whose tilts are of completely different size.



One overshoots, the other has not started



Both move by about as much

$$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla L(w)$$

$$v \leftarrow \beta_2 v + (1 - \beta_2) \nabla L(w)^2$$

$$w \leftarrow w - \alpha m / (\sqrt{v} + \delta)$$

m is the direction and is exactly momentum. v is the size that knob has been showing. δ keeps the division safe; the paper calls it ϵ and this lecture has spent that letter on the window. The published rule also bias corrects m and v , which matters only for the first few steps.

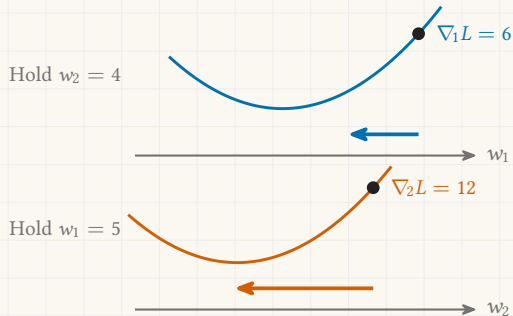
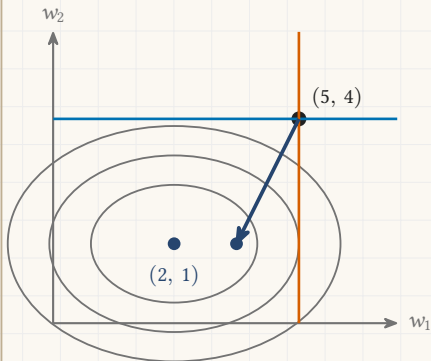
6

Section 6

More than one knob

Two knobs

$L(w_1, w_2) = (w_1 - 2)^2 + 2(w_2 - 1)^2 + 3$, seen from above.



$$\begin{aligned}(w_1, w_2) &\leftarrow (5, 4) - 0.25 \cdot (6, 12) \\ &= (3.5, 1)\end{aligned}$$

The gradient in d dimensions

One window per knob. Open them one at a time and hold the others still.

Knob	Window	Left	Right	Tilt	Written
w_1	$w_1 \pm \frac{\varepsilon}{2}$	a_1	b_1	$\frac{b_1 - a_1}{\varepsilon}$	$\begin{bmatrix} \nabla_1 L(w) \\ \nabla_2 L(w) \\ \vdots \\ \nabla_d L(w) \end{bmatrix}$
w_2	$w_2 \pm \frac{\varepsilon}{2}$	a_2	b_2	$\frac{b_2 - a_2}{\varepsilon}$	
$\vdots$				$\vdots$	
w_d	$w_d \pm \frac{\varepsilon}{2}$	a_d	b_d	$\frac{b_d - a_d}{\varepsilon}$	

a_j and b_j are the heights at the two edges of knob j 's own window, with every other knob held where it is.

$$\nabla L(w) = (\nabla_1 L(w), \dots, \nabla_d L(w))$$

Gradient descent in d dimensions

The same rule. The guess is a list, the gradient is a list, and they subtract term by term.

$$w \leftarrow w - \alpha \nabla L(w), \quad w \in \mathbb{R}^d$$

Coordinate by coordinate, that is d separate copies of the one dimensional rule:

```
for j in range(d):  
    a = L(w_shift(w, j, -eps/2))    # move knob j back by eps/2  
    b = L(w_shift(w, j, +eps/2))    # move knob j on by eps/2  
    grad[j] = (b - a) / eps  
w = [w[j] - alpha * grad[j] for j in range(d)]
```

That costs $2d$ evaluations of L per step, and for a neural network d runs to millions.

The right way is backpropagation, which is the 14:00 lecture.

Neural networks

Foundations of AI

Carlos Cotrini

UZH CAS Gamechanger AI

26 September 2026

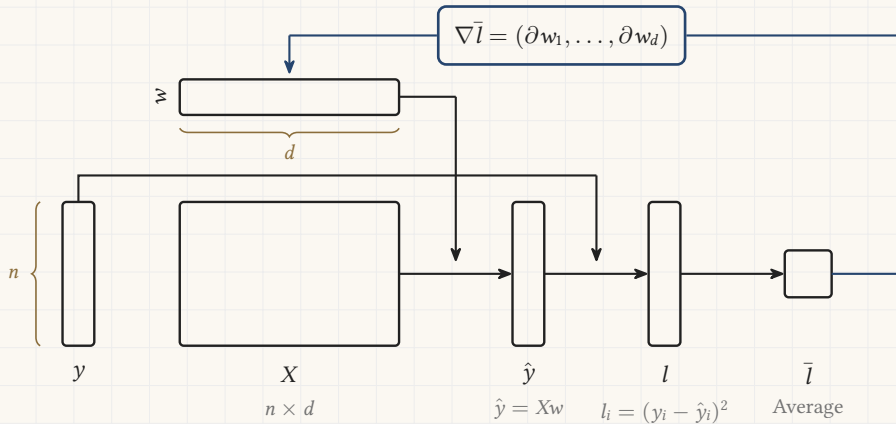
1

Section 1

Linear regression in one picture

Linear regression with gradient descent

The whole of this morning, drawn once. Each box is a piece of data, and its shape is the shape of the data.



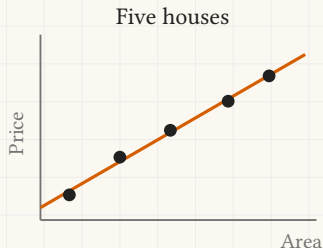
2

Section 2

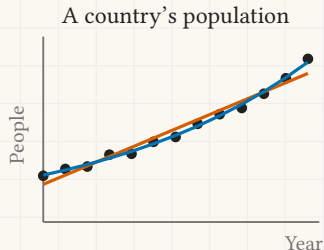
What a linear model cannot fit

A linear model does not fit everything

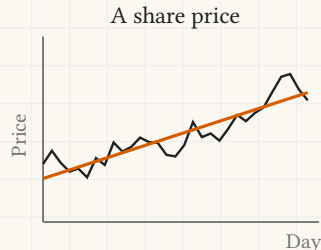
Three sets of measurements, and the best straight line through each.



The largest miss is 83'413 CHF, which is 3.8 percent of the price range.



The straight line misses by 12.4 percent of the range. The curve $y = e^{wx}$ misses by 3.2 percent. Synthetic data.



The straight line misses by 32.8 percent of the range, and no elementary curve does better. Synthetic data.

3

Section 3

Universal approximators

Universal approximators

Let K be a closed and bounded set of inputs, and let $\mathcal{F}$ be a family of functions from K to the real numbers.

$\mathcal{F}$ is a **universal approximator** on K if for every continuous $g : K \rightarrow \mathbb{R}$ and every $\varepsilon > 0$ there is an $f \in \mathcal{F}$ with $\max_{x \in K} |g(x) - f(x)| < \varepsilon$.

Two families are known to meet this definition.

- 1 The polynomials, on a closed interval. That is the Weierstrass approximation theorem, and it dates from 1885.
- 2 The neural networks. The rest of this lecture builds one, and the last slide states the theorem.

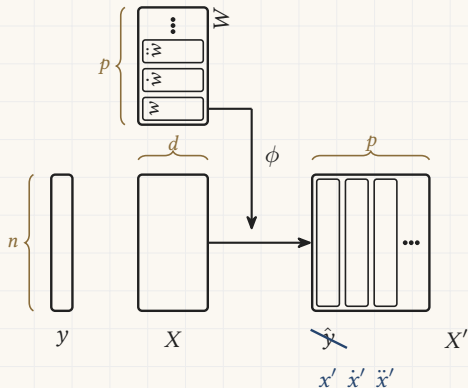
4

Section 4

Neural networks

From one weight vector to a layer

The same X , and more than one weight vector. Every block is drawn to scale.

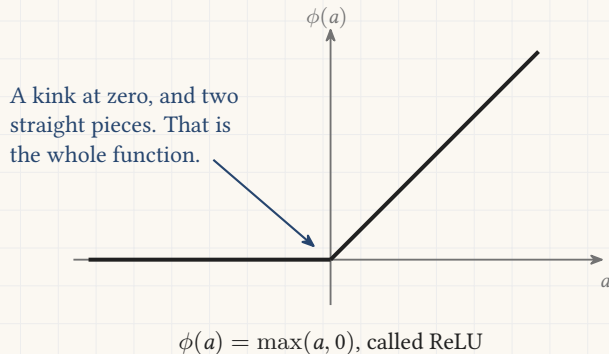


$$X' = \phi(X W)$$

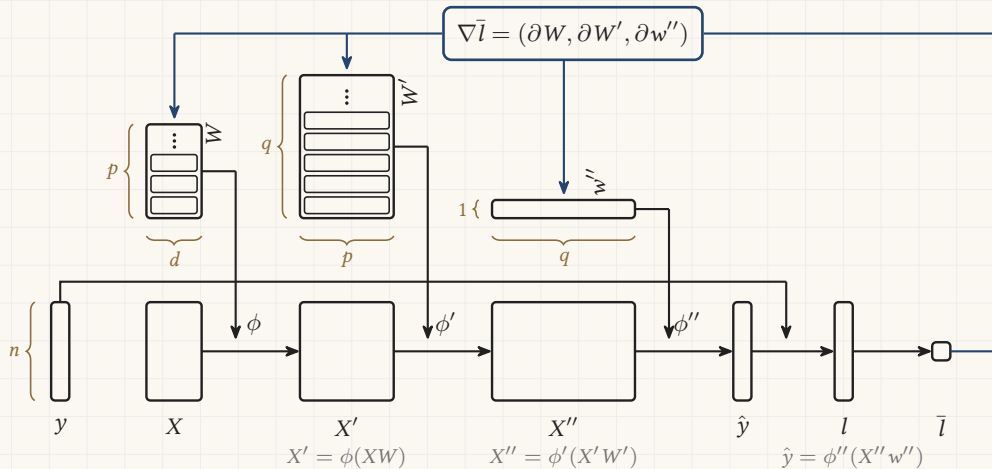
W is laid on its side, so one of its columns is a row up there. Each row makes one column of X' , and the two are drawn at the same pitch, so you can count them off against each other. W has d rows, which is what makes the product defined, and p columns, which is a number you choose.

The activation function

ϕ is applied to every entry on its own, and it is the only piece of the network that is not a matrix product.

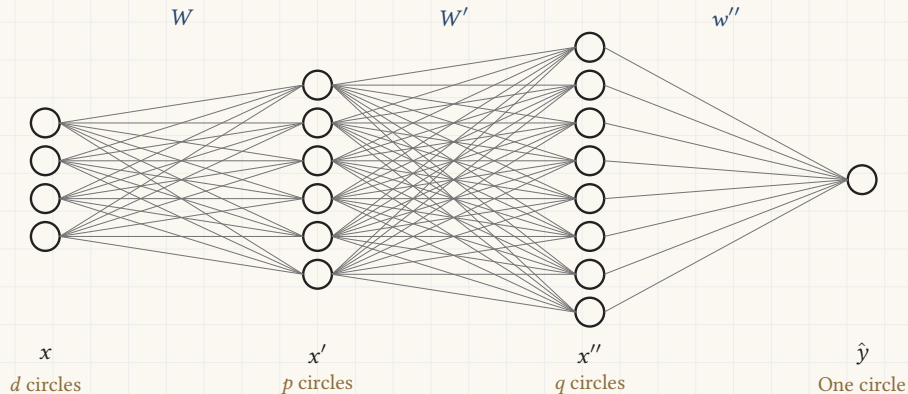


A neural network



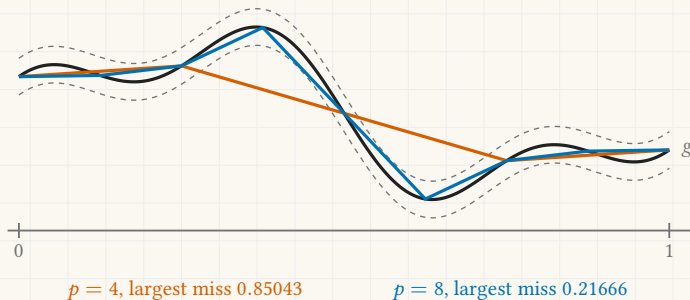
The same network, drawn as neurons

One circle for every column of every weight block.



The universal approximation theorem

A continuous target, and networks with p hidden units drawn over it. The band is the accuracy $\varepsilon = 0.25$.



A network with one hidden layer, and enough units in it, is a universal approximator.

The largest miss falls to 0.06944 at $p = 16$, to 0.01788 at $p = 32$ and to 0.00448 at $p = 64$.

Cybenko, 1989. Hornik, Stinchcombe and White, 1989.

Try it: TensorFlow Playground

Pick the spiral dataset, then add neurons and layers until the two colours separate.

<https://playground.tensorflow.org>

